

Sustentación de Pruebas: LocalWork

Este documento contiene las respuestas a las preguntas planteadas en el PDF, basadas en la arquitectura real de nuestro proyecto LocalWork.

1. Estudio previo y Selección

→ ¿Qué funcionalidad crítica existe en su proyecto?

El "core business" de LocalWork es la intermediación laboral. Por tanto, las dos funcionalidades más críticas son:

- El **Sistema de Autenticación y Autorización**: Si los usuarios no pueden iniciar sesión o los roles (Seeker vs Employer) se mezclan, el sistema pierde su propósito y expone datos.
- El **Flujo de Postulación a Empleos**: Que un trabajador pueda buscar empleos con filtros complejos (por IA, por ubicación, por categoría) y pueda aplicar correctamente.

→ ¿Dónde está el mayor riesgo de regresión?

El mayor riesgo de regresión está en la capa de servicios (`applications.service.ts` y `jobs.service.ts`). Por ejemplo, la lógica de validación de estados (impedir que alguien retire una postulación que ya fue aceptada).

2. Pruebas Unitarias Elegidas (Backend)

*Ubicadas en `backend/src/tests/unit/` *

- `auth.service.test.ts` (Login y Registro)
- `applications.service.test.ts` (Gestión de postulaciones)

→ ¿Por qué es unitaria y no de integración?

Porque en estas pruebas **aislamos el servicio de la base de datos**. Usamos `vi.mock` sobre nuestro wrapper de Supabase. Estamos probando pura lógica de negocio y

condicionales (ej. ¿Se lanza un error 404 si la postulación no existe?), sin depender de la red, los controladores o el enrutador de Express.

3. Pruebas de Integración Elegidas (Backend)

Ubicadas en `backend/src/tests/` (archivos raíz del test suite)

- `auth.test.ts` (Endpoints de `/api/auth`)
- `jobs.test.ts` (Endpoints de `/api/jobs`)

→ ¿Por qué es de integración y no unitaria?

Porque a diferencia de las unitarias, aquí utilizamos `supertest` para levantar la aplicación Express completa y hacemos peticiones HTTP reales (`GET /api/jobs`). Estas pruebas validan que el Router, los Middlewares (ej. Autenticación, Zod validation) y el Controlador trabajan en armonía. Seguimos usando una base de datos de prueba / mock global, pero **todo el pipeline HTTP** está siendo probado.

4. Pruebas E2E Elegidas (Frontend)

Ubicadas en `frontend/e2e/` (Playwright)

- `auth.spec.ts` (Flujo de Inicio de Sesión y Dashboard)
- `seeker.spec.ts` (Búsqueda de Empleos y Filtros)

→ ¿Por qué elegimos estas E2E?

Porque representan los "Happy Paths" más críticos del usuario final. Usamos **Playwright** porque nos permite levantar un navegador real (Chromium), interactuar con el DOM, hacer clic en botones e interceptar la red, simulando la experiencia exacta de nuestros usuarios en Aguachica.

5. Justificación General y Hallazgos

→ ¿Qué riesgo cubre cada prueba?

- **Unitarias:** Cubren riesgos de cálculo, validación de estados incorrectos (ej. retirar una postulación en estado "Completado").
- **Integración:** Cubren riesgos de seguridad (¿Funciona el Middleware de autenticación y de RLS?) y comunicación de red.
- **E2E:** Cubren riesgos de usabilidad y renderizado en el cliente (¿Se muestra correctamente el botón tras buscar?).

→ ¿Qué descubrieron al escribirlas? ¿Encontraron bugs?

Sí. Escribir la prueba unitaria `withdraw` (retirar postulación) nos hizo darnos cuenta de que un buscador de empleo podía borrar su postulación incluso si el empleador ya lo había marcado como "Contratado" o "Finalizado". Esto rompía el flujo del empleador. Al descubrirlo con la prueba, agregamos un "guard" que ahora rechaza la acción con un error 409 si el estado no es "pending".

→ ¿Cómo las ejecutan? ¿Están en el README?

Sí, todas las instrucciones de ejecución están claramente definidas en el archivo `README.md` en la raíz del repositorio.

- **Backend (Unit/Integration):** `npm test` dentro de la carpeta `backend/`.
- **Frontend (E2E):** `npx playwright test` dentro de la carpeta `frontend/`.
- Usamos una filosofía de "**Criterio sobre Cantidad**". Tenemos 194 pruebas en el backend de rápida ejecución y 2 pruebas E2E robustas para validar los flujos principales en el frontend.